

TABLE OF CONTENTS

How to Auto-Generate your Providers with Flutter Riverpod Generator	1
What we will cover	1
Should we write providers by hand?	2
The @riverpod annotation	4
A new Riverpod syntax?	5
Getting Started with riverpod_generator	6
Starting the code generator in "watch" mode	6
Creating the first annotated provider	7
Creating a provider for a Repository class	9
Creating and reading an annotated FutureProvider	10
How are generated families implemented?	12
Does this work with StreamProvider too?	12
Mixing the old and new syntax	13
Using autoDispose vs keepAlive	14
Riverpod Generator: Tradeoffs	15
Advantage: auto-generate the right kind of provider	15
Advantage: autoDispose by default	16
Advantage: stateful hot-reload for providers	16
Disadvantage: code generation	16
Disadvantage: Not all provider types are supported yet	18
Sample apps with source code	18
1. TMDB movies app	19
2. Time Tracker app with Firebase	20
Conclusion	21
Flutter Foundations Course Now Available	22
Flutter Foundations Course	23
Want More?	23
Flutter In Production	23
Flutter & Firebase Masterclass	23
The Complete Dart Developer Guide	24
Flutter Animations Masterclass	24



How to Auto-Generate your Providers with Flutter Riverpod Generator

#dart

#flutter

#state-management

#riverpod



Andrea Bizzotto

Updated Mar 7, 2023 · 18 min read



Source code on GitHub

Riverpod is a powerful **reactive caching and data-binding framework** for Flutter.

It gives us many different kinds of providers that we can use to:

- access dependencies in our code (with `Provider`)
- cache asynchronous data from the network (with `FutureProvider` and `StreamProvider`)
- manage local application state (with `StateProvider`, `StateNotifierProvider`, and `ChangeNotifierProvider`)

But writing a lot of providers by hand can be error-prone, and choosing which provider to use is not always easy. 🤔

What if I told you that you no longer have to?

What if you could simply annotate your code with `@riverpod`, and let `build_runner` generate all the providers **on the fly**?

As it turns out, that's what the new riverpod_generator package is for (and it can make our life a **lot easier**).

What we will cover

1

There is much to cover, so I'll break this into two articles.

In this first article, we'll learn how to **generate providers from functions** using the new `@riverpod` syntax.

As part of this, I'll show you how to:

- declare providers with the `@riverpod` syntax
- convert `FutureProvider` to the new syntax
- pass arguments to a provider, overcoming the limitations of the old `family` modifier

And in the next article, we'll learn how to **generate providers from classes**, and see on how to **completely replace** `StateNotifierProvider` and `StateProvider` with the new `Notifier` and `AsyncNotifier` classes.

You can find the second article here: [How to use Notifier and AsyncNotifier with the new Flutter Riverpod Generator](#).

Update: a **third article** is also available, showing how to [boost your workflow with Riverpod Lint & Riverpod Snippets](#).

We'll also cover some tradeoffs, so you can decide if you should use the new syntax in your own apps.

Ready? Let's go! 🙌

This article assumes that you're already familiar with Riverpod. If you're new to Riverpod, read: [Flutter Riverpod 2.0: The Ultimate Guide](#)

Should we write providers by hand?

This is a good question.

On one hand, you may have simple providers such as this:

```
// a provider for the Dio client to be used by the rest of the app
final dioProvider = Provider<Dio>((ref) {
  return Dio();
});
```

On the other hand, some providers have dependencies and may take an argument using the `family` modifier:

```
// a provider to fetch the movie data for a given movie id
final movieProvider = FutureProvider.autoDispose
  .family<TMDBMovie, int>((ref, movieId) {
    return ref
      .watch(moviesRepositoryProvider)
      .movie(movieId: movieId);
  });
```

And if you have a `StateNotifierProvider` with the `family` modifier, the syntax becomes even more complex as you have to specify **three** type annotations:

```
final emailPasswordSignInControllerProvider =
StateNotifierProvider.autoDispose
  .family<
    EmailPasswordSignInController, // the StateNotifier subclass
    EmailPasswordSignInState, // the type of the underlying state
    class
    EmailPasswordSignInFormType // the argument type passed to the
    family
    >((ref, formType) {
    return EmailPasswordSignInController(
      authRepository: ref.watch(authRepositoryProvider),
      formType: formType,
    );
  });
```

While the static analyzer can help us figure out how many types we need, the code above is not very readable.

Is there a simpler way? 🤔

The @riverpod annotation

Let's consider this `FutureProvider` once again:

```
// a provider to fetch the movie data for a given movie id
final movieProvider = FutureProvider.autoDispose
    .family<TMDBMovie, int>((ref, movieId) {
    return ref
        .watch(moviesRepositoryProvider)
        .movie(movieId: movieId);
});
```

The essence of this provider is that we can use it to fetch a movie by calling this method:

```
// declared inside a MoviesRepository class
Future<TMDBMovie> movie({required int movieId});
```

But what if rather than creating the provider above, we could write something like this?

```
@riverpod
Future<TMDBMovie> movie(
    MovieRef ref, {
    required int movieId,
}) {
    return ref
        .watch(moviesRepositoryProvider)
        .movie(movieId: movieId);
}
```

This is in line with how we define a function:

- return type first
- then function name
- then list of arguments
- then function body

```

@riverpod
Future<TMDBMovie> movie(
  MovieRef ref, {
    required int movieId,
  }) {
  return ref
    .watch(moviesRepositoryProvider)
    .movie(movieId: movieId);
}

```

Return type

Function name

List of arguments

Function body

A function definition. 1: return type, 2: function name, 3: list of arguments, 4: function body

This is more intuitive than declaring a `FutureProvider.family<TMDBMovie, int>`, with the return type **next to** the argument type.

A new Riverpod syntax?

When Remi presented the [new Riverpod syntax during Flutter Vikings](#), I was a bit confused.

But having tried it on some of my projects, I've grown to like its simplicity.

The new API is much more streamlined and brings **two significant usability improvements**:

- you no longer have to worry about which provider to use
- you can pass named or positional arguments to a provider as you please (just like you would do with any function)

This is a **big leap forward** for Riverpod itself, and learning the new API will make your life much easier.

So let me show you how it all works.

Rather than starting from scratch, we'll take some providers from an existing app and convert them to the new syntax. At the end, I'll share an example repo with the full source code.

Getting Started with riverpod_generator

As explained in the [riverpod_generator](#) page on pub.dev, we need to add these packages to

`pubspec.yaml`:

```
dependencies:
  # or flutter_riverpod/hooks_riverpod as per
  https://riverpod.dev/docs/getting_started
  riverpod:
  # the annotation package containing @riverpod
  riverpod_annotation:

dev_dependencies:
  # a tool for running code generators
  build_runner:
  # the code generator
  riverpod_generator:
  # riverpod_lint makes it easier to work with Riverpod
  riverpod_lint:
  # import custom_lint too as riverpod_lint depends on it
  custom_lint:
```

Note how I've also added the `riverpod_lint` and `custom_lint` packages. To learn more about what `riverpod_lint` can do, read: [How to boost your dev workflow with Riverpod Lint & Riverpod Snippets](#).

Starting the code generator in "watch" mode

Then, we need to run this command on the terminal:

```
dart run build_runner watch -d
```

The `-d` flag is optional and is the same as `--delete-conflicting-outputs`. As the name implies, it ensures that we override any conflicting outputs from previous builds (which is normally what we want).

This will **watch all the Dart files** in our project and automatically update the generated code when we make changes.

So let's start creating some providers. 📌

Creating the first annotated provider

As a first step, let's consider this simple provider:

```
// dio_provider.dart
import 'package:dio/dio.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

// a provider for the Dio client to be used by the rest of the app
final dioProvider = Provider<Dio>((ref) {
  return Dio();
});
```

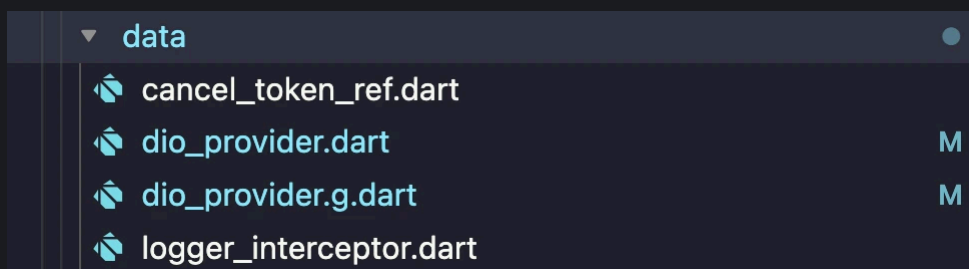
Here's how we should modify this file to use the new syntax:

```
import 'package:dio/dio.dart';
// 1. import the riverpod_annotation package
import 'package:riverpod_annotation/riverpod_annotation.dart';

// 2. add a part file
part 'dio_provider.g.dart';

// 3. use the @riverpod annotation
@riverpod
// 4. update the declaration
Dio dio(DioRef ref) {
  return Dio();
}
```

As soon as we save this file, `build_runner` will get to work and produce `dio_provider.g.dart` in the same folder:



A dio_provider.g.dart file is generated alongside our Dart file

The new `.g.dart` files are generated **alongside** the existing ones, so you don't need to change your folder structure at all.

And if we open the generated file, this is what we see:

```
// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'dio_provider.dart';

//
*****
// RiverpodGenerator
//
*****

String _$dioHash() => r'26723d20a4ee2d05c3b01acad1196ed96cece567';

/// See also [dio].
@ProviderFor(dio)
final dioProvider = AutoDisposeProvider<Dio>.internal(
  dio,
  name: r'dioProvider',
  debugGetCreateSourceHash:
    const bool.fromEnvironment('dart.vm.product') ? null : _$dioHash,
  dependencies: null,
  allTransitiveDependencies: null,
);

typedef DioRef = AutoDisposeProviderRef<Dio>;
// ignore_for_file: unnecessary_raw_strings, subtype_of_sealed_class,
invalid_use_of_internal_member, do_not_use_environment,
prefer_const_constructors, public_member_api_docs,
avoid_private_typedef_functions
```

What is most relevant is that this file:

- contains the `dioProvider` that we need (with additional properties that we can use for debugging)
- defines a `DioRef` type as `AutoDisposeProviderRef<Dio>`

This means that we only need to write this code:

```
part 'dio_provider.g.dart';
```

```
@riverpod
Dio dio(DioRef ref) {
  return Dio();
}
```

And riverpod_generator will create the corresponding `dioProvider` and the `DioRef` type that we pass as an argument to our function.

All providers created with riverpod_generator use the `autoDispose` modifier by default. If you're not familiar with this, read about the autoDispose modifier.

Creating a provider for a Repository class

Now that we have a `dioProvider`, let's try to use it somewhere.

For example, suppose we have a `MoviesRepository` class that defines some methods for fetching movies data:

```
class MoviesRepository {
  MoviesRepository({required this.client, required this.apiKey});
  final Dio client;
  final String apiKey;

  // search for movies that match a given query (paginated)
  Future<List<TMDBMovie>> searchMovies({required int page, String query = ''});
  // get the "now playing" movies (paginated)
  Future<List<TMDBMovie>> nowPlayingMovies({required int page});
  // get the movie for a given id
  Future<TMDBMovie> movie({required int movieId});
}
```

To create a provider for this repository, we can write this:

```
part 'movies_repository.g.dart';
```

```
@riverpod
```

```

MoviesRepository moviesRepository(MoviesRepositoryRef ref) =>
MoviesRepository(
    client: ref.watch(dioProvider), // the provider we defined above
    apiKey: Env.tmdbApiKey, // a constant defined elsewhere
);

```

As a result, `riverpod_generator` will create a `moviesRepositoryProvider` and the `MoviesRepositoryRef` type for us.

When creating a provider for a `Repository`, don't add the `@riverpod` annotation to the `Repository` class itself. Instead, create a **separate global function** that returns an instance of that `Repository` and **annotate that**. We will learn more about using `@riverpod` with classes in the next article.

Creating and reading an annotated FutureProvider

As we have seen, given a `FutureProvider` such as this:

```

// a provider to fetch the movie data for a given movie id
final movieProvider = FutureProvider.autoDispose
    .family<TMDBMovie, int>((ref, movieId) {
    return ref
        .watch(moviesRepositoryProvider)
        .movie(movieId: movieId);
});

```

We can convert it to use the `@riverpod` annotation:

```

@riverpod
Future<TMDBMovie> movie(
    MovieRef ref, {
    required int movieId,
}) {
    return ref
        .watch(moviesRepositoryProvider)
        .movie(movieId: movieId);
}

```

And watch it inside our widget:

```
class MovieDetailsScreen extends ConsumerWidget {
  const MovieDetailsScreen({super.key, required this.movieId});
  final int movieId;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // movieId is a *named* argument
    final movieAsync = ref.watch(movieProvider(movieId: movieId));
    return movieAsync.when(
      error: (e, st) => Text(e.toString()),
      loading: () => CircularProgressIndicator(),
      data: (movie) => SomeMovieWidget(movie),
    );
  }
}
```

This is the most important part:

```
// movieId is a *named* argument
final movieAsync = ref.watch(movieProvider(movieId: movieId));
```

As we can see, `movieId` is a **named argument** because we have defined it as such in the `movie` function:

```
@riverpod
Future<TMDBMovie> movie(
  MovieRef ref, {
    required int movieId,
  }) {
  return ref
    .watch(moviesRepositoryProvider)
    .movie(movieId: movieId);
}
```

This means that we're no longer restricted to defining a provider family with only one positional argument.

In fact, we don't even care that we're using a family at all.

All we do is define a function with a "ref" object and as many named or positional arguments as we like, and riverpod_generator takes care of the rest.

How are generated families implemented?

If we're curious and take a peek at how the `movieProvider` has been generated, we can find this:

```
typedef MovieRef = AutoDisposeFutureProviderRef<TMDBMovie>;

@ProviderFor(movie)
const movieProvider = MovieFamily();

class MovieFamily extends Family<AsyncValue<TMDBMovie>> {
  const MovieFamily();

  MovieProvider call({
    required int movieId,
  }) {
    return MovieProvider(
      movieId: movieId,
    );
  }
  ...
}
```

This uses callable classes - a neat feature of the Dart language that allows us to call `movieProvider(movieId: movieId)` rather than `movieProvider.call(movieId: movieId)`.

Does this work with StreamProvider too?

As we have seen, using `@riverpod` makes it easy to generate a `FutureProvider`.

And since Riverpod Generator 2.0.0, streams are supported too.

In fact, if we have a method that returns a `Stream`, we can create the corresponding provider like this:

```
@riverpod
Stream<int> values(ValuesRef ref) {
    return Stream.fromIterable([1, 2, 3]);
}
```

This is possible thanks to the new `StreamNotifier` class that was introduced in Riverpod 2.3, which I have covered more in detail here:

- [New in Riverpod 2.3: StreamNotifier](#)

Streams and `StreamProvider` are quite useful if we use a realtime database such as Cloud Firestore, or if we're communicating with a custom backend that supports web sockets, so it's nice that they're now supported by Riverpod Generator. 🍌

`StateNotifier` and `ChangeNotifier` are not supported by the new generator, so you can't convert existing code using `StateNotifierProvider` and `ChangeNotifierProvider` to the new syntax yet. But you can generate providers based on the `Notifier` and `AsyncNotifier` classes, as I have explained [in this article](#).

Mixing the old and new syntax

Let's revisit this function once again:

```
@riverpod
Future<TMDBMovie> movie(
    MovieRef ref, {
    required int movieId,
}) {
    return ref
        .watch(moviesRepositoryProvider)
        .movie(movieId: movieId);
}
```

Note how inside it, we call `ref.watch(moviesRepositoryProvider)`.

But are we allowed to use a provider based on the old syntax inside an auto-generated provider?

As it turns out, the new `Riverpod Lint` package introduces a new lint rule called `avoid_manual_providers_as_generated_provider_dependency`. And if we don't follow this rule, we get this warning:

Generated providers should only depend on other generated providers. Failing to do so may break rules such as "provider_dependencies"

So if we plan to migrate our code, it is best to start from the providers that **don't depend on other providers**, and make our way through the provider-tree until all providers are updated.



Using `autoDispose` vs `keepAlive`

A common requirement is to destroy a provider's state when it is no longer used.

With the old syntax, this was done with the `autoDispose` modifier (which was **disabled by default**).

If we use the new `@riverpod` syntax, `autoDispose` is now **enabled by default** and has been renamed to `keepAlive`.

This means that we can write this:

```
// keepAlive is false by default
@riverpod
Future<TMDBMovie> movie(MovieRef ref, {required int movieId}) {
  ...
}
```

Which is equivalent to this:

```
// keepAlive: false is the same as using autoDispose
@Riverpod(keepAlive: false)
Future<TMDBMovie> movie(MovieRef ref, {required int movieId}) {
  ...
}
```

And the generated `movieProvider` will be disposed when no longer used.

On the other hand, if we set `keepAlive` to `true`, the provider will remain "alive":

```
// keepAlive: true is the same as *NOT* using autoDispose
@Riverpod(keepAlive: true)
Future<TMDBMovie> movie(MovieRef ref, {required int movieId}) {
  ...
}
```

Note that if you want to obtain a `KeepAliveLink` to implement some custom caching behaviour, you can still do so inside the provider:

```
@riverpod
Future<TMDBMovie> movie(MovieRef ref, {required int movieId}) {
  // get the [KeepAliveLink]
  final link = ref.keepAlive();
  // start a 60 second timer
  final timer = Timer(const Duration(seconds: 60), () {
    // dispose on timeout
    link.close();
  });
  // make sure to cancel the timer when the provider state is disposed
  ref.onDispose(() => timer.cancel());
  return ref
    .watch(moviesRepositoryProvider)
    .movie(movieId: movieId);
}
```

For more details on how to customise the caching behaviour using `keepAlive`, read: [Caching with Timeout](#).

Riverpod Generator: Tradeoffs

Now that we've learned about the new syntax and how the generator works, let's summarize the pros and cons.

Advantage: auto-generate the right kind of provider

The biggest advantage is that we no longer have to figure out which kind of provider we need (`Provider` vs `FutureProvider` vs `StreamProvider` etc.) since the code generator will figure it out from the **function signature**.

The new `@riverpod` syntax also makes it easy to declare **complex providers** that take one or more arguments (such as the `FutureProvider.family` we have seen above).

Another bonus is that the generated code creates a new **specialized** type for each "ref" object, and this can be easily inferred from the function name:

- `moviesRepository()` → `moviesRepositoryProvider` and `MoviesRepositoryRef`
- `movie()` → `movieProvider` and `MovieRef`

This makes runtime type errors less likely since **our code won't compile if we don't use the correct types** in the first place.

Advantage: autoDispose by default

With the new syntax, all the generated providers use `autoDispose` by default.

This is a sensible choice since we shouldn't hold on to the state of providers that are no longer in use.

And as I have explained in my [Riverpod 2.0 guide](#), we can tweak the dispose behaviour by calling `ref.keepAlive()` and even implement a [timeout-based caching strategy](#) if needed.

Advantage: stateful hot-reload for providers

The package documentation says this:

When modifying the source code of a provider, on hot-reload Riverpod will re-execute that provider **and only that provider**.

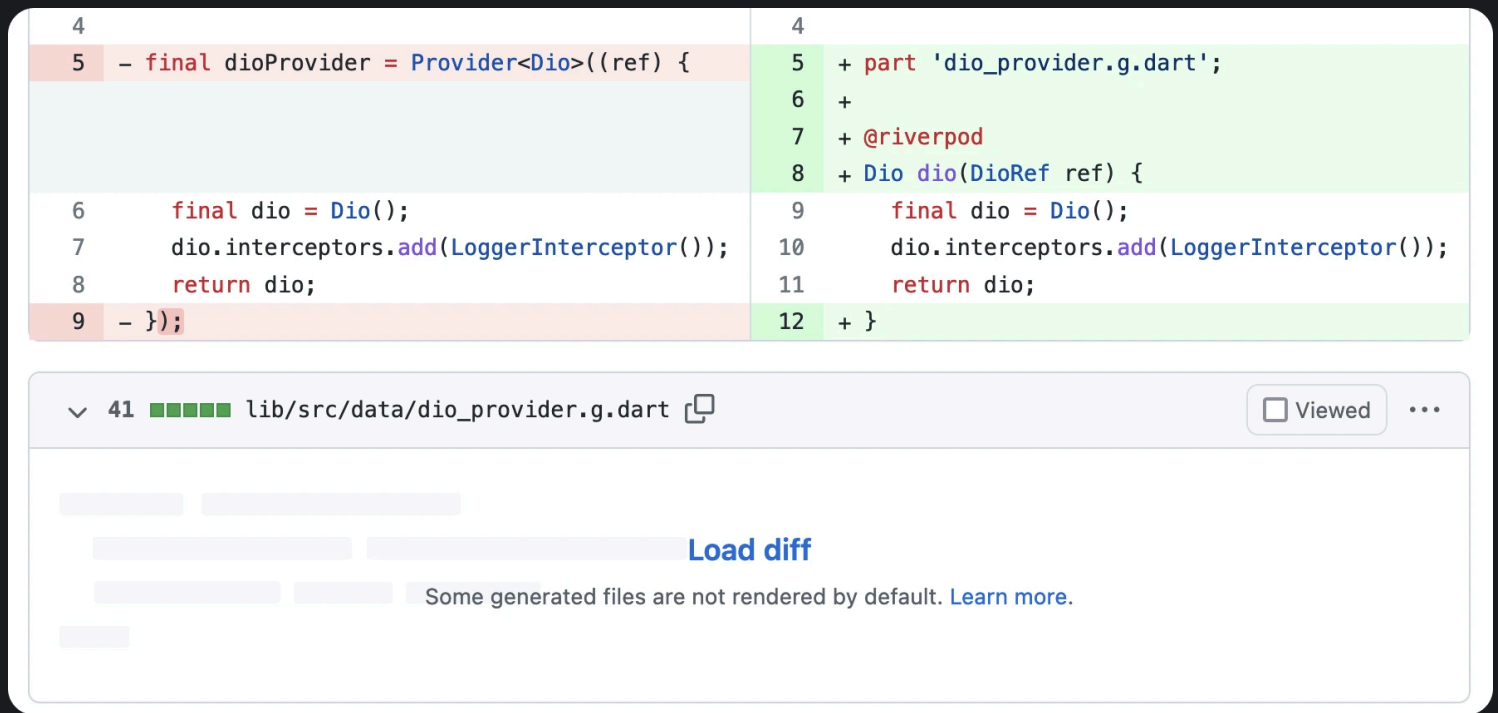
That's a welcome improvement. 😊

Disadvantage: code generation

The drawbacks of Riverpod Generator all boil down to one thing: **code generation**.

Even the simplest provider produces 15 lines of generated code in a separate file, which can slow down the build process and clutter our project with extra files.

If we add the generated files to version control, they will show up on Pull Requests every time they change:



GitHub diff showing the auto-generated provider file

If we don't want that, we can add `*.g.dart` to `.gitignore` to exclude all generated files from our repository.

This has two implications:

- other team members need to always be running `dart run build_runner watch -d` during development
- CI build workflows need to run the code generator before compiling the app (leading to longer builds that cost you more build minutes)

In practice, I've observed that `dart run build_runner watch -d` is fast (at least on small projects) and produces **sub-second updates after the first build**:

```
[INFO] -----  
[INFO] Starting Build  
[INFO] Updating asset graph completed, took 0ms  
[INFO] Running build completed, took 309ms
```

```
[INFO] Caching finalized dependency graph completed, took 12ms
[INFO] Succeeded after 323ms with 4 outputs (16 actions)
```

This is in line with the response times from hot-reload and makes the development workflow very smooth. 🍑

However, you'll need a beefy development machine if you want to use `build_runner` on bigger projects.

And since CI build minutes are not free, I recommend adding all the generated files to version control (along with `.lock` files to ensure everyone runs with the same package versions).

Disadvantage: Not all provider types are supported yet

Out of the eight different kinds of providers, `riverpod_generator` only supports the following:

1. `Provider`
2. `FutureProvider`
3. `StreamProvider`
4. `NotifierProvider` (new in Riverpod 2.0)
5. `AsyncNotifierProvider` (new in Riverpod 2.0)

Legacy providers like `StateProvider`, `StateNotifierProvider`, and `ChangeNotifierProvider` are not supported, and I've already explained how they can be replaced in my article about how to use `Notifier` and `AsyncNotifier` with the new Flutter Riverpod Generator.

And with the introduction of the Riverpod Lint package, adopting the new `@riverpod` syntax becomes much easier.

So whether your app uses a realtime database and relies heavily on **streams**, or talks to REST API using **futures**, you can already benefit from the new generator.

Sample apps with source code

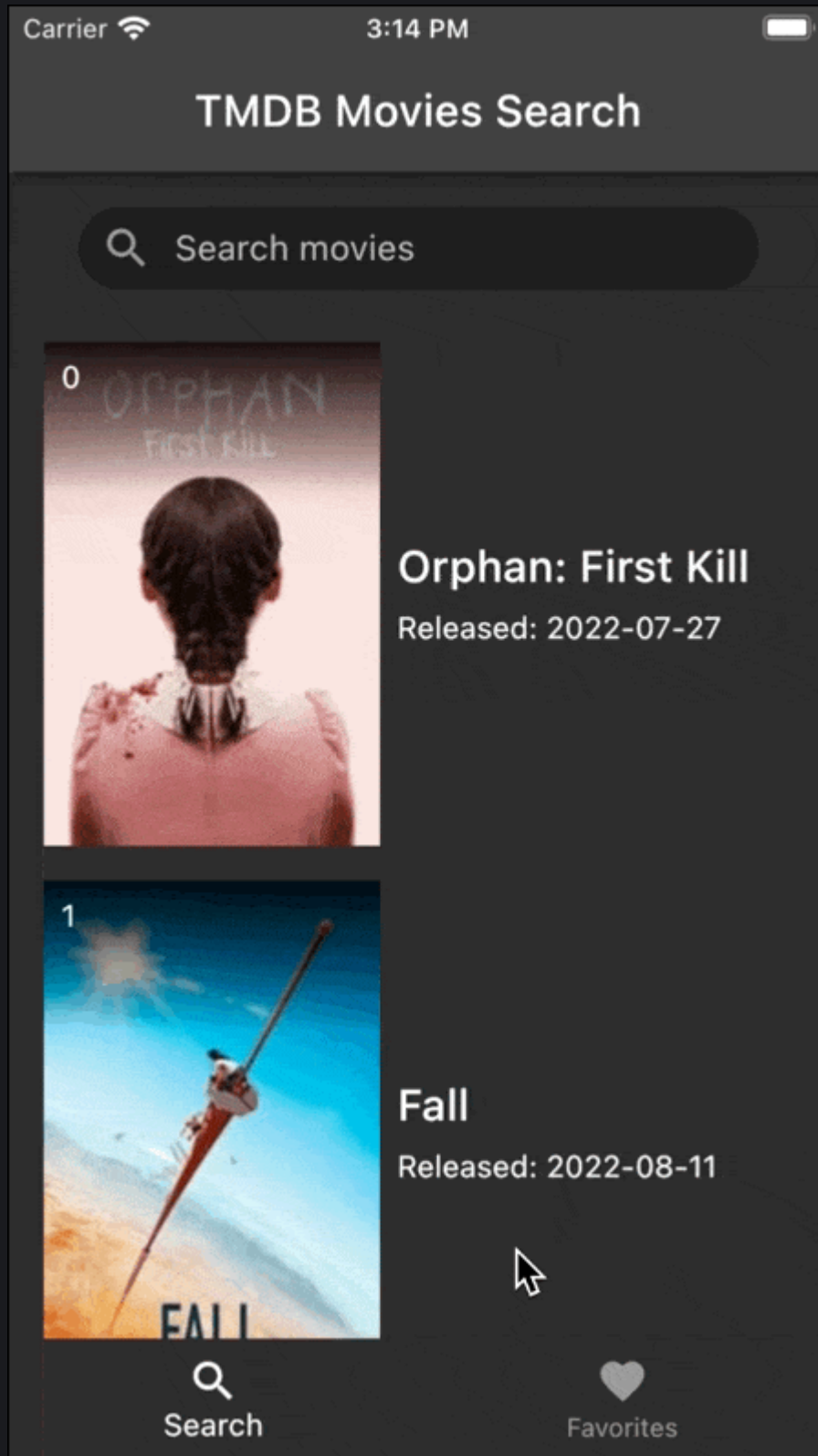
So far, we have seen how to create providers with the new `@riverpod` syntax.

And if you're wondering how this all fits together in real-world apps, I have good news for you.

In fact, two of my open-source Flutter apps already use the new Riverpod Generator. 📌

1. TMDB movies app

The first one is a movies app based on the TMDB APIs:



TMDB Movies app with Riverpod

This app includes support for:

- infinite scrolling with pagination
- pull to refresh
- search functionality

All these features are built natively with Riverpod (without external packages).

And since the app already uses Freezed for JSON serialization, adding the riverpod_generator package seemed a great fit.

The source code includes things we haven't covered here, such as how to cancel network requests with `CancelToken` from the dio package.

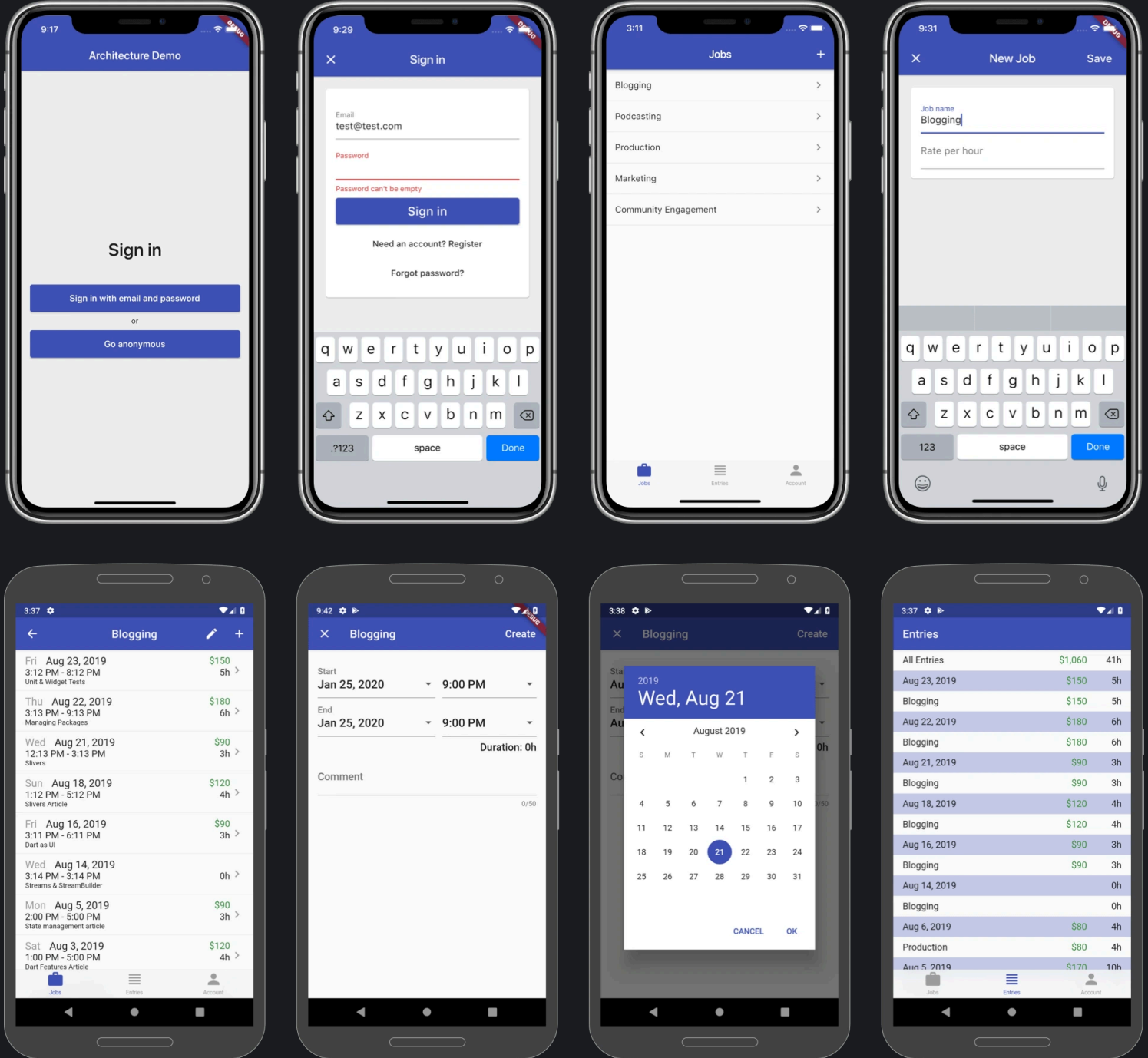
This app is still WIP, and I'll try to add more features in the future.

But you can already check it out here: 📌

- [TMDB Movies app with Riverpod](#)

2. Time Tracker app with Firebase

The second one is a time-tracking app built with Flutter & Firebase:



Time tracking app with Flutter & Firebase

The source code is updated to all the latest Riverpod packages and you can find it here:

- [Time Tracking app with Flutter & Firebase](#)

If you want, you can even see how I've migrated the code to the new Riverpod syntax [on this PR](#). 🍌

Conclusion

As we have seen, the [riverpod_generator](#) package has a lot to offer. Here are a few reasons to use it:

- automatically generate the right kind of provider

- much easier to create providers with arguments, overcoming the limitations of the "old" `family` modifier syntax
- **improved type safety** and fewer type errors at runtime
- `autoDispose` by default

However, some legacy provider types are not supported.

And since the new package relies on code generation, you have to:

- deal with additional auto-generated files in the project
- decide if the generated files should be added to git and plan accordingly

If you're on the fence about code generation, consider [this take](#) by Remi Rousselet:

Generated code isn't "boilerplate". You don't really care about the generated code. It's not really meant to be read or edited. It's there for the compiler, not for developers. In fact, you can hide it from your IDE explorer, and you typically don't commit generated files.

Overall, the most significant advantage is **improved developer productivity**.

Using the new syntax means that you need to learn and use a **smaller and more familiar API**. And this makes Riverpod more approachable to developers that were confused by the old APIs.

But just to be clear: `riverpod_generator` is an **optional package** built on top of `riverpod` and the "old" syntax is not going away any time soon.

And because the new Riverpod syntax is **compatible** with the old one, you can **adopt it incrementally** when migrating providers in your codebase.

So why not give it a try? 🕶

Flutter Foundations Course Now Available

I launched a brand new course that covers state management with Riverpod in great depth, along with other important topics like app architecture, routing, testing, and much more:



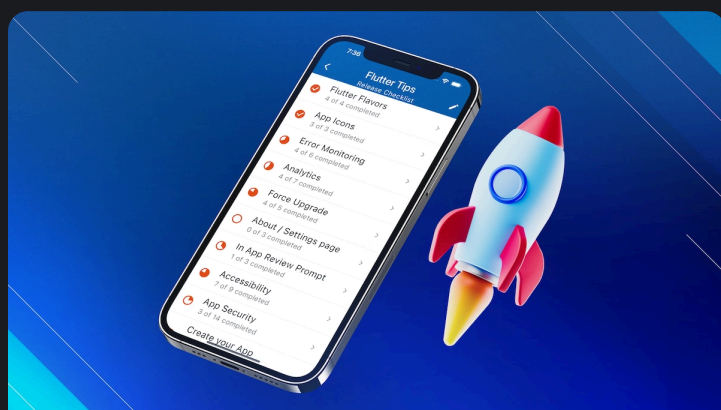
Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much more by building a Flutter eCommerce app on iOS, Android, and web.

Want More?

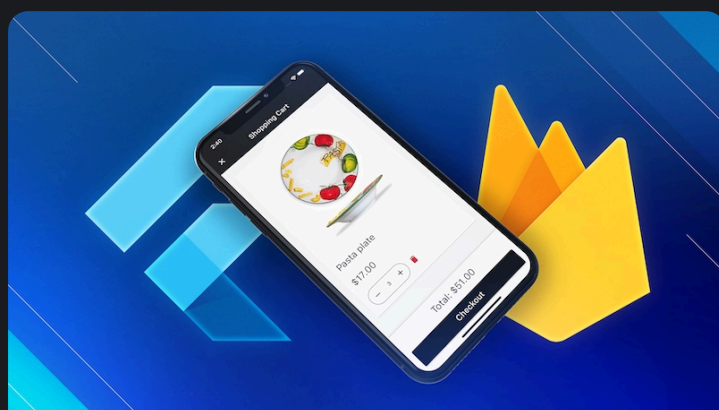
Invest in yourself with my high-quality Flutter courses.



Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀



Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app with Flutter & Firebase.



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.



CODE WITH ANDREA

Copyright © 2020-present Coding With Flutter Limited

Contact

Twitter

Discord

GitHub

RSS

Meta

Privacy Policy

